TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

Tập 7 · 16 bài Codeforces, dịch tiếng Việt

TOÁN TRÊN BIT VÀ XOR



Cơ sở XOR

1 0 1 1 0

0 1 1 0 0

0 0 0 1 1

hạng r = 3 → 2^{n-r} tập con

Lê Thành Nam

$$a + b = (a \oplus b) + 2 \cdot (a \,\&\, b)$$

Tính chất XOR · AND · OR · Xét từng bit · Cơ sở tuyến tính XOR

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

LÊ THÀNH NAM

Tổng hợp và biên soạn

MỤC LỤC

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

Giới thiệu **2**

Chương 1. Tính chất của phép XOR **3**

Bài 1. AGAGA XOOORRR *(Codeforces 1516B)* 4
Bài 2. Ehab và thuật XOR *(Codeforces 1325D)* 8
Bài 3. Phương trình XOR *(Codeforces 627A)* 11
Bài 4. Cô bé và XOR lớn nhất *(Codeforces 276D)* 14

Chương 2. Phép AND và phép OR **17**

Bài 5. Đá và đòn bẩy *(Codeforces 1420B)* 18
Bài 6. AND, OR và tổng bình phương *(Codeforces 1368D)* 21
Bài 7. Xóa mảng *(Codeforces 1601A)* 24
Bài 8. AND khác 0 *(Codeforces 1615B)* 28

Chương 3. Xét từng bit **31**

Bài 9. Johnny và đợt tụt hạng mới *(Codeforces 1362C)* 32
Bài 10. Moamen và XOR *(Codeforces 1557C)* 35
Bài 11. Tổng hàm XOR *(Codeforces 1879D)* 39

Chương 4. Cơ sở tuyến tính XOR **42**

Bài 12. Tập con XOR khác 0 *(Codeforces 1100G)* 43
Bài 13. Mahmoud, Ehab và bài XOR nữa *(Codeforces 959F)* 46
Bài 14. Ivan và bánh mì kẹp *(Codeforces 1100F)* 50
Bài 15. Bài toán đường đi ngắn nhất? *(Codeforces 845G)* 54
Bài 16. Tập con chính phương *(Codeforces 895C)* 58

Lê Thanh Nam

GIỚI THIỆU

Tài liệu này dành cho ai

Đây là Tập 7 của bộ Tuyển tập bài tập Toán trên Codeforces, gồm 16 bài về toán trên bit và phép XOR, sắp thành 4 chương: *tính chất của phép XOR; phép AND và phép OR; xét từng bit; cơ sở tuyến tính XOR*. Các bài tiêu biểu: 1420B Rock and Lever; 1362C Johnny and Another Rating Drop; 1516B AGAGA XOOORRR; 1325D Ehab the Xorcist; 959F Mahmoud and Ehab and yet another xor task; 1100F Ivan and Burgers. Tài liệu phù hợp với học sinh đã quen biểu diễn nhị phân và các phép toán bit cơ bản, đang luyện thi học sinh giỏi cấp tỉnh, cấp quốc gia và các kì thi tương đương.

Mỗi bài gồm hai phần. Đề bài được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm bảng subtask do tài liệu bổ sung. Hướng dẫn giải đi qua 4 bước tư duy; mỗi bước có code chạy được, có hình minh họa các bit, bài nào cũng kết thúc bằng bảng so sánh các tầng lời giải đo thật trên máy.

4 bước tư duy giải một bài

Bước	Nội dung
Ngây thơ	Ý tưởng tự nhiên nhất, làm đúng như đề mô tả; có code và độ phức tạp.
Bế tắc	Biểu đồ thời gian đo thật, chỉ ra chỗ chậm hoặc chỗ sai, kết thúc bằng một câu hỏi then chốt.
Phá bế tắc	Nhận xét hoặc tính chất giúp vượt qua chỗ tắc, có ví dụ và hình minh họa.
Tối ưu	Code cuối cùng, độ phức tạp và các lưu ý: kiểu dữ liệu, tràn số, đọc ghi nhanh.

Lê Thành Nam

Subtask và bảng so sánh các tầng

Đề Codeforces không chia subtask. Tài liệu bổ sung cho mỗi bài ba mức: Subtask 1 (30%) là ràng buộc nhỏ để cách ngây thơ lấy điểm, Subtask 2 (30%) và Subtask 3 (40%) là ràng buộc gốc hoặc ràng buộc mở rộng (ví dụ tới 10^9 thay vì vài nghìn) để thấy rõ vai trò của từng định lí.

Cuối mỗi bài là bảng so sánh: mỗi tầng code chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm. Đầu mỗi bài ghi giới hạn thời gian gốc của Codeforces; riêng bảng so sánh dùng chung một giới hạn 1 giây cho mọi bài để dễ so sánh. Một tầng chỉ được điểm của subtask nếu cho kết quả đúng trong không quá 1 giây, dùng không quá 512 MB bộ nhớ và 8 MB ngăn xếp (ô "sai" là chương trình cho kết quả sai, ô "lỗi" là chương trình bị dừng vì tràn bộ nhớ hoặc tràn ngăn xếp). Thời gian đo bằng g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả đọc và ghi dữ liệu, lấy lần nhanh hơn trong hai lần chạy.

Sử dụng tài liệu

Với mỗi bài, hãy tự suy nghĩ ít nhất 20–30 phút trước khi đọc hướng dẫn. Khi đọc, dừng lại ở câu hỏi then chốt cuối phần **Bế tắc** và tự trả lời trước khi xem phần **Phá bế tắc**.

Mọi code trong tài liệu đã được biên dịch, chạy đúng mọi ví dụ của đề, và đối chiếu với lời giải tham chiếu viết riêng bằng Python trên 150–300 test ngẫu nhiên (với bài có nhiều đáp án, dùng bộ kiểm tra đáp án). Code được viết theo phong cách thi đấu, không chú thích, tên biến đầy đủ, dễ đọc.

Chương 1. Tính chất của phép XOR

Trọng tâm chương

Phép XOR ($\oplus$) là phép cộng không nhớ theo từng bit. Ba tính chất làm nên mọi lời giải: $x \oplus x = 0$, $x \oplus 0 = x$, giao hoán và kết hợp. Từ đó: XOR của các phần bằng XOR của toàn bộ, XOR có "tiền tố" như phép cộng, và liên hệ quan trọng $a + b = (a \oplus b) + 2(a \,\&\, b)$.

Kiến thức cần nhớ

- Triệt tiêu. $x \oplus x = 0$: số chẵn bản sao biến mất.
- Tiền tố. XOR đoạn $[l, r] = P_r \oplus P_{l-1}$.
- Tổng và XOR. $a + b = (a \oplus b) + 2(a \,\&\, b)$; tổng ≥ XOR và cùng tính chẵn lẻ.
- Bit cao nhất. Muốn XOR lớn, tham lam từ bit cao xuống.
- Đếm theo bit. Biết $a \oplus b$ và $a \,\&\, b$ thì mỗi bit của (a, b) có 1 hoặc 2 khả năng.

Các bài trong chương

Bài	Tên bài	Mã Codeforces	Kĩ thuật chính
1	AGAGA XOOORRR	1516B	XOR toàn mảng + tham lam
2	Ehab và thuật XOR	1325D	Tổng = XOR + 2 · AND
3	Phương trình XOR	627A	Đếm theo bit
4	Cô bé và XOR lớn nhất	276D	Bit cao nhất khác nhau

Lê Thành Nam

Bài 1. AGAGA XOOORRR

Codeforces 1516B – AGAGA XOOORRR Thời gian: 1 giây · Bộ nhớ: 256 MB
Đề gốc: https://codeforces.com/problemset/problem/1516/B

Đề bài

Cho mảng a gồm n số nguyên không âm. Thao tác: Chọn hai phần tử kề nhau, xóa cả hai và đặt vào chỗ đó một phần tử bằng XOR của chúng (mảng ngắn đi 1).

Hỏi có thể thực hiện một số thao tác (có thể không thao tác nào) để mảng còn ít nhất 2 phần tử và mọi phần tử bằng nhau hay không.

Dữ liệu vào

Dòng đầu chứa số test t.

Mỗi test gồm hai dòng: Dòng đầu chứa n, dòng thứ hai chứa $a_1, a_2, \ldots, a_n$.

Ràng buộc chung: $1 \le t \le 15$; $2 \le n$; $0 \le a_i < 2^{30}$.

Kết quả

Với mỗi test, in "YES" hoặc "NO".

Ví dụ

Dữ liệu vào	Kết quả
2 3 0 2 2 4 2 3 1 10	YES NO

Giải thích ví dụ

Test 1: Gộp 2 và 2 được 0, mảng còn [0, 0].

Subtask

Subtask	Ràng buộc	Điểm
Subtask 1	n ≤ 100	30
Subtask 2	n ≤ 2 000 (ràng buộc gốc)	30
Subtask 3	n ≤ 10^6	40

Hướng dẫn giải

NGÂY THƠ Thử mọi cách cắt thành 2 hoặc 3 đoạn

Mỗi phần tử cuối cùng là XOR của một đoạn liên tiếp của mảng ban đầu. Vậy câu hỏi là: Có chia được mảng thành ít nhất 2 đoạn liên tiếp có XOR bằng nhau không?

Chỉ cần thử 2 hoặc 3 đoạn: Nếu chia được thành k ≥ 4 đoạn cùng XOR v thì gộp 3 đoạn đầu (XOR v ⊕ v ⊕ v = v) được k - 2 đoạn, lặp lại tới khi còn 2 hoặc 3 đoạn.

Cách ngây thơ thử mọi vị trí cắt, tính XOR từng đoạn bằng vòng lặp.

Lê Thành Nam

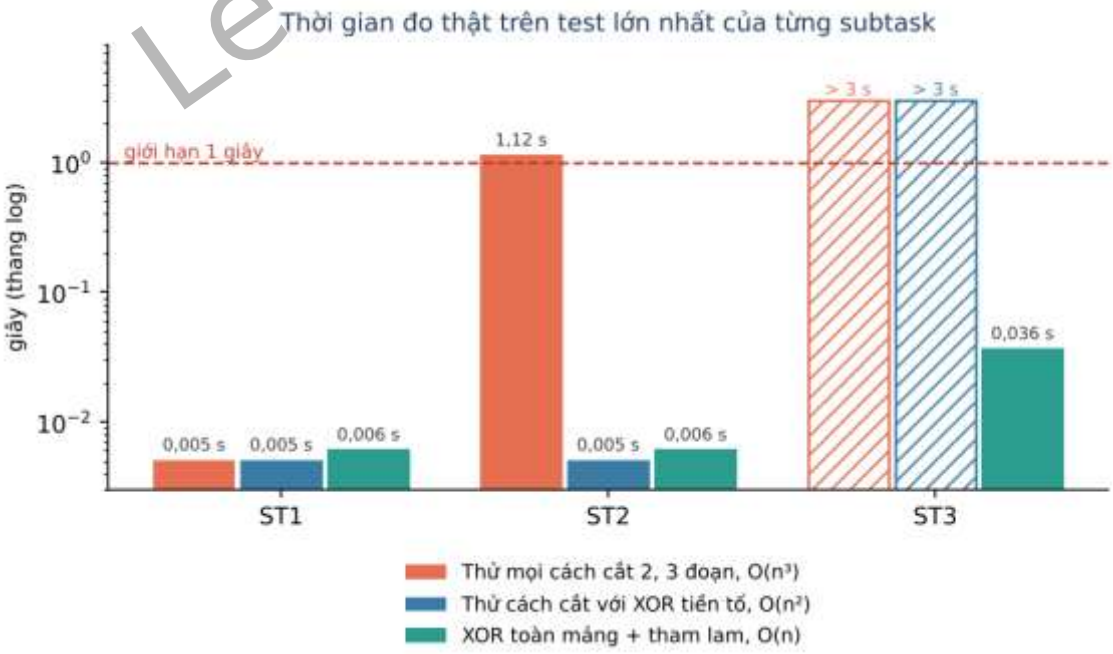
Code. Tầng 1. Thử mọi cách cắt 2, 3 đoạn, $O(n^3)$

```
#include <bits/stdc++.h>
using namespace std;
int readInt() {
    int c = getchar_unlocked(), x = 0;
    while (c < '0' || c > '9') c = getchar_unlocked();
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar_unlocked(); }
    return x;
}
int segmentXor(const vector<int> &a, int from, int to) {
    int value = 0;
    for (int i = from; i < to; i++) value ^= a[i];
    return value;
}
int main() {
    int testCount = readInt();
    while (testCount--) {
        int n = readInt();
        vector<int> a(n);
        for (int i = 0; i < n; i++) a[i] = readInt();
        bool possible = false;
        for (int i = 1; i < n && !possible; i++)
            if (segmentXor(a, 0, i) == segmentXor(a, i, n)) possible = true;
        for (int i = 1; i < n && !possible; i++)
            for (int j = i + 1; j < n && !possible; j++) {
                int first = segmentXor(a, 0, i), second = segmentXor(a, i, j), third =
segmentXor(a, j, n);
                if (first == second && second == third) possible = true;
            }
        puts(possible ? "YES" : "NO");
    }
}
```

Độ phức tạp: $O(n^3)$.

BẾ TẮC n^3 phép tính

n = 2 000 đã mất 1,1 giây (Hình 1.1). Câu hỏi then chốt: XOR của cả mảng cho biết gì về XOR của các đoạn?



Hình 1.1. *Thời gian đo thật của ba tầng lời giải.*

Lê Thành Nam

PHÁ BẾ TẮC Phá bế tắc: XOR toàn mảng

Ba tính chất cơ bản của XOR: $x \oplus x = 0$, $x \oplus 0 = x$, và có giao hoán, kết hợp. Vì vậy XOR các đoạn bằng XOR toàn mảng T.

• Chia thành 2 đoạn cùng XOR v: $T = v \oplus v = 0$. Ngược lại nếu $T = 0$ thì cắt ở bất kì đâu, hai đoạn có XOR bằng nhau. Vậy $T = 0$ là đủ.

• Chia thành 3 đoạn cùng XOR v: $T = v \oplus v \oplus v = v$, tức là mỗi đoạn có XOR bằng T.

Tầng giữa dùng XOR tiền tố P (XOR đoạn [i, j) bằng $P_j \oplus P_i$) để thử 2 và 3 đoạn trong $O(n^2)$: Tức thì với $n = 2\,000$, nhưng không kịp với $n = 10^6$.

Code. Tầng 2. Thử cách cắt với XOR tiền tố, $O(n^2)$

```
#include <bits/stdc++.h>
using namespace std;
int readInt() {
    int c = getchar_unlocked(), x = 0;
    while (c < '0' || c > '9') c = getchar_unlocked();
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar_unlocked(); }
    return x;
}
int main() {
    int testCount = readInt();
    while (testCount--) {
        int n = readInt();
        vector<int> prefix(n + 1, 0);
        for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] ^ readInt();
        bool possible = false;
        for (int i = 1; i < n && !possible; i++)
            if (prefix[i] == (prefix[n] ^ prefix[i])) possible = true;
        for (int i = 1; i < n && !possible; i++)
            for (int j = i + 1; j < n && !possible; j++) {
                int first = prefix[i], second = prefix[j] ^ prefix[i], third =
prefix[n] ^ prefix[j];
                if (first == second && second == third) possible = true;
            }
        puts(possible ? "YES" : "NO");
    }
}
```

TỐI ƯU Tham lam cắt đoạn có XOR bằng T

Khi $T \neq 0$, đi từ trái sang, cộng dồn XOR; mỗi khi bằng T thì cắt và đặt lại về 0. Nếu cắt được ít nhất 3 đoạn thì phần còn lại sau đoạn thứ 3 có XOR bằng 0 (vì nếu cắt đúng 3 đoạn, phần thừa có XOR $T \oplus T \oplus T \oplus T = 0$; nhiều hơn thì gộp như trên) và gộp được vào đoạn cuối (Hình 1.2). Cắt tham lam sớm nhất không bao giờ làm mất cơ hội cắt về sau.

$O(n)$: 0,036 giây với $n = 10^6$.



Hình 1.2. *Ba đoạn cùng XOR bằng XOR toàn mảng.*

Lê Thanh Nam